Radical Denesting after `StradFixed`

Safety boundaries, mathematical certificates, and a hardened reference implementation

Prepared for Vladimir Reshetnikov

September 5, 2026

Principal finding. Certification must occur where a numerical subexpression is replaced, not only when the entire input happens to be an algebraic number. The recovered wrapper lets a custom solver cross this boundary without a local equality check.

Scope limitation. The current repository directories and the initial safety/wrapper portion of `StradFixed.wl` were retrieved. Subsequent access failed before the rest of the source, the current report bodies, or their complete execution logs could be read. Findings identify the inspected functions and do not invent line numbers or defects in the unavailable backend. The accompanying code is an *independently written replacement design*, not a source-complete patch.

Validation. The accompanying Python run passed 156 exact mathematical checks and nine lexical/structural checks. Thirty-one native Wolfram regression tests are supplied, but *were not executed*: the connected Wolfram service failed before evaluation and no local kernel was available. The replacement therefore requires native validation before production adoption.

Archive contents. This article in PDF and LaTeX; `StradImproved.wl`; Wolfram regression tests and original-source probes; exact-check Python source and results; a source-access ledger and usage guide.

Abstract

A corrected denester can have a sound exact-number core and still expose an unsound public transformation boundary. This review separates those questions. It identifies a source-level acceptance gap for symbolic hosts, an implicit-assumptions contract problem, incomplete forwarding of option defaults, and resource limits that do not cover the entire recovered wrapper. A second branch issue is demonstrated for the common-factor helper's internal representation: positivity of an aggregate factor does not justify distributing a fractional power over its individually recorded complex factors. The latter is a helper-level counterexample, not a demonstrated wrong public numerical result; the visible numerical certification layer is an important defense.

The article proves the polynomial-GCD and roots-of-unity reconstruction principles, derives local norm certificates, and explains why degree reduction, exact equality, and radical-depth improvement are different objectives. An independent reference implementation replaces markers and unrestricted symbolic cleanup with local exact acceptance, a restricted expression grammar, shared resource controls, explicit outcome reporting, and bounded bottom-up passes. Its mathematical design is analyzed separately from its unexecuted Wolfram runtime behavior. Source-access and validation limits are recorded throughout so that neither earlier repository tests nor independent SymPy checks are mistaken for a complete native audit of the present code.

Contents

1 Evidence, scope, and the correct interpretation of this review

1.1 What was actually available

The requested repository organizes a literature report, downloaded literature, a unified review of earlier code, and a corrected Wolfram package in separate directories [2, 3, 4, 1]. That organization is valuable: a claim about a mathematical algorithm, a claim made by a review, and an observation from a kernel run are different kinds of evidence.

The initial raw-source response exposed the package declarations and options, the algebraicity and equality predicates, cost functions, denominator rationalization, `Factorc`, `fullfactor`, `combine`, the `Strad/DenestRadicals` wrapper, `recursiveCore`, `normalizeRadicand`, and the first `rationalQ` definition introducing the next section. GitHub reported 504 physical source lines; the browser's text representation used a different line count. Only its initial portion was readable. All subsequent continuation and download routes failed. In particular, the complete `DenestCore` implementation and the remaining multiplier machinery were not available for inspection.

The current report and unified-review directory descriptions were inspected. A prior Library version of the unified literature report and an earlier review of the original algorithm supplied background, but are *not* assumed to be byte-identical to the current repository versions. Relevant primary papers and official language documentation were inspected separately; this is not a claim to have read every file in the literature directory.

Material	Evidence obtained	Limit on conclusions
Corrected package	Recovered safety, factoring, and wrapper definitions	No complete backend audit, source snapshot, or commit hash.
Current unified review	Directory description and reported prior native testing	Its reported kernel runs were not repeated here; modular report bodies and full logs were unavailable.
Literature	Selected primary texts, including BFHT, Gkioulekas, Jeffrey–Rich, and Honsbeek	Theorems are used within their stated models, not as evidence that this package implements them completely.
This delivery	Exact Python computations, lexical/static checks, compiled PDF	No Wolfram parser, native execution, performance benchmark, or full compatibility claim.

The repository review description reports testing with Wolfram 15.0.1 on Windows. That is a useful provenance statement from the repository, not an execution result of this investigation [4]. It would be equally misleading to ignore that reported work or to inherit its success claims for newly written code.

1.2 Evidence labels used below

A **static finding** follows from recovered control flow or data flow. A **mathematical counterexample** disproves a transformation rule under explicitly stated inputs. A **conditional risk** identifies an unproved assumption or a plausible failure mechanism without asserting that a particular native run produced it. A **coverage limitation** describes an intentionally or accidentally unsupported

transformation, not an incorrect value. Proposed test outputs are labeled as expectations, not transcripts.

The highest-confidence public-interface finding is the unchecked symbolic-host solver boundary. The strongest helper-level branch finding is the aggregate-factor counterexample. Performance claims are deliberately more modest: there are structural opportunities to remove unnecessary work, but no native speedup factor has been measured.

1.3 What should not be inferred

Nothing here establishes that every earlier defect remains in `StradFixed`. The visible source already incorporates several important repairs. Nor does an uninspected backend justify a claim that its multiplier ordering, stale-state handling, termination guards, or cap arithmetic are still wrong. Those questions require the exact missing source and its tests. The accompanying access ledger records that distinction in a machine-independent form.

2 Executive assessment and priorities

The package should be treated as a *heuristic expression optimizer with exact numerical acceptance*, not a universal minimum-depth denesting decision procedure. Its central algebraic construction is mathematically credible. The main remaining concern is that the wrapper does not consistently enforce the same acceptance policy for every supported input form.

ID	Priority	Finding	Evidence type
F1	High	Custom solver results may enter a symbolic host without local equality certification.	Static acceptance gap.
F2	High	Whole-host simplification inherits ambient assumptions while the interface does not record a conditional result.	Static contract gap.
F3	High, latent	Aggregate positivity does not validate distribution over recorded complex factors in `combine`.	Helper counterexample; public reachability unproved.
F4	Medium	A core budget does not bound wrapper preprocessing, callbacks, repeated certifications, lists, and recursion together.	Static resource scope.
F5	Medium	Delegation forwards explicit option rules rather than all effective entry-point defaults.	Static configuration gap.
F6	Medium	Approximate rejection and Boolean failure collapse obscure unknown equality outcomes.	Reliability risk, not a shown false positive.
F7	Medium	The score lacks an admissible-output grammar and differs from its usage description.	Objective/API issue.
F8	Medium	Disabling the outer factor pass does not bypass factoring inside normalization.	Option-scope ambiguity.
F9	Low–medium	All-or-nothing rollback after `ToRadicals` can discard useful local conversions.	Coverage limitation.
F10	Design	Marker traversal, bounded dependency resolution, and recursive re-entry need explicit invariants and outcome reporting.	Maintenance/test obligation, not a proved loop.

The first release priority is not a more elaborate multiplier generator. It is one invariant: *every*

accepted numerical replacement has been certified against the exact numerical expression it replaces. A fast incomplete search with this invariant is useful. An impressive search that occasionally bypasses it is not a dependable simplifier.

3 A precise mathematical and software contract

3.1 Values, expressions, and branches

An expression denotes a selected value, not merely a polynomial equation. Throughout this review, Wolfram `Power` means the principal complex power. For nonzero z and rational p,

$$z^p = \exp(p \operatorname{Log} z), \qquad -\pi < \operatorname{Arg} z \le \pi.$$

A real odd root is a different operation on a negative real argument. For example,

$$(1 - \sqrt{2})^5 = 41 - 29\sqrt{2} < 0,$$

but $1 - \sqrt{2}$ is not the principal complex fifth root of the number on the right. Wolfram's documented distinction between `Power` and real-root operations is therefore an input specification, not a cosmetic convention [10].

The preservation requirement is

$$\text{value}(E_{\text{out}}) = \text{value}(E_{\text{in}}) \tag{3.1}$$

under the same branch and domain convention. A common minimal polynomial does not establish (3.1). For instance, $\sqrt{3} + \sqrt{2}$ and $\sqrt{3} - \sqrt{2}$ both satisfy $X^4 - 10X^2 + 1 = 0$, but differ by $2\sqrt{2}$.

3.2 Depth and presentation cost

For rational constants and field operations, define syntactic depth by

$$\text{depth}(q) = 0, \qquad \text{depth}(A \circ B) = \max\{\text{depth}(A), \text{depth}(B)\}, \qquad \text{depth}(A^{p/q}) = 1 + \text{depth}(A)$$

when p/q is reduced and $q > 1$. Integer powers are field operations for this purpose. A `Root` object is an opaque algebraic representation, not proof that a radical has depth zero in a radical-expression model. A cost function can penalize it separately, but must explain that policy.

The replacement uses the lexicographic tuple

$$(\text{opaque algebraic objects},\ \text{depth},\ \text{rational-power nodes},\ \text{integer bit cost},\ \text{leaf count}). \tag{3.2}$$

The bit-cost component distinguishes a short integer from one with thousands of digits, something plain leaf count does not do. This remains a chosen preference, not a canonical definition of simplicity. In particular, the initial opaque-object penalty can favor a larger radical expression over a compact `Root`; explicit size limits constrain that preference.

The implementation-oriented measure in Jeffrey–Rich is different again: its treatment of sums is additive rather than a maximum over parallel branches [7]. A proof of progress under one measure must not be advertised as progress under another.

3.3 An admissible syntax is part of the optimization problem

If arbitrary special functions are allowed as output atoms, a syntactic root counter can be evaded by a change of notation. A robust contract specifies both the objective and the language over which it is optimized. The replacement admits exact rational/Gaussian-rational constants, algebraic arithmetic, rational powers, and recognized algebraic `Root`/`AlgebraicNumber` objects. Other heads are not accepted as numerical proposals merely because `NumericQ` recognizes them.

This is intentionally conservative. It excludes some perfectly valid algebraic constants written with trigonometric or other functions. The benefit is that the meaning of an accepted "improvement" is inspectable. A later preprocessing module may translate such constants into the admitted language, provided that translation has its own certificate.

3.4 The symbolic-host congruence principle

Proposition 3.1 (Local certificates suffice for pure hosts)**.** *Let a host expression be built from symbols and numerical leaves using fixed, single-valued scalar operations and lists. Replace finitely many numerical leaves or numerical subexpressions by expressions denoting exactly the same selected values. Wherever the original host is defined, the reconstructed host has the same value.*

Proof. Proceed by structural induction. An unchanged symbol or leaf has the same value, and a certified replacement has the same value by hypothesis. At an internal node, the induction hypothesis supplies the same ordered argument values to the same operation; its selected value is therefore unchanged. For a list, apply this argument componentwise. No continuity assumption is required. Even a discontinuous principal branch returns the same result on *identical* arguments. Undefined denominators and undefined powers are not converted into defined values by substituting equal arguments. □

This proposition explains why local certification is enough for $x + \alpha$ even though the full expression is not an algebraic number. It does *not* justify transformations of arbitrary Wolfram programs. A head may inspect syntax, have holding attributes, invoke side effects, or use custom upvalues. The replacement therefore traverses only `List`, `Plus`, `Times`, and rational/integer `Power`, and assumes ordinary pure scalar semantics. Unknown heads remain opaque.

4 What the recovered corrected wrapper already does well

Several visible changes should be preserved. Private package symbols avoid accidental interference with common user variables. The wrapper does not globally redefine printing. Its numerical acceptance compares against the original input rather than merely a reconstructed principal root. Strict cost improvement rejects many expansions that do not help. The normalization routine reconstructs a value and verifies equality to the original radicand before wrapping it. These are meaningful defenses [1].

The use of exact algebraic arithmetic is also appropriate. `RootReduce` is documented to canonicalize the supported exact algebraic combinations; comparing its difference result with literal zero is a reasonable trusted-kernel acceptance test on that domain [11]. The fallback setting `Method -> "ExactAlgebraics"` in `PossibleZeroQ` is not the ordinary approximate zero heuristic: the documentation specifically gives an exact guarantee for explicit algebraic numbers [12]. It would be incorrect to flag that setting itself as an approximate-proof defect.

Likewise, roots-of-unity enumeration is a strength, not a branch error. It is precisely the device that avoids assuming multiplicativity of principal fractional powers. Section 8 proves this carefully. The remaining question is where the resulting candidate is checked and what kind of expression is allowed to receive it.

5 Correctness boundaries and branch-sensitive helpers

5.1 F1: A numerical callback crosses an unchecked symbolic boundary

The recovered wrapper obtains proposed solutions for marked numerical problems from a configurable solver. Its final equality filter is conditional on the *whole original input* passing the algebraicity predicate. For a symbolic host this filter is skipped, and cost comparison remains as the acceptance criterion [1].

Consider the source-level probe

```
a = Sqrt[5 + 2 Sqrt[6]];
RadicalDenest`DenestRadicals[
  x + a, "Solver" -> Function[z, 0]
]
```

The outer radical is an exact numerical target. Replacing its marker by the solver result produces x. The entire original expression $x + \alpha$ is not numeric, so the whole-input equality filter does not protect this substitution. The cost of x is lower because the radical nodes have disappeared. Thus x becomes an eligible unchecked final candidate, even though

$$x - (x + \sqrt{5 + 2\sqrt{6}}) = -\sqrt{5 + 2\sqrt{6}} \neq 0.$$

The expected revealing output of the probe is x; it was not executed here. The static defect is already the existence of this acceptance path, independently of any further behavior in the unavailable final cleanup helper.

For an entirely numerical input, the visible final certification is an important backstop and should reject zero. This contrast localizes the problem: it is not evidence that `RootReduce` is unsound, nor that the mathematical GCD construction is wrong. It is a missing check between a callback and the surrounding expression.

The same boundary must reject approximate numbers, failure symbols, unevaluated solver calls, unrelated parameters, and other malformed proposals. A callback being user-supplied does not make its mathematical result a certificate. Conversely, treating its result as untrusted does not make arbitrary callback *code* safe to execute; a Wolfram simplifier is not a sandbox.

Required repair. Retain the original numerical target α with every proposal. Check the proposal's syntax and exactness, certify equality to α, and only then rebuild the host. A final whole-number check can remain as defense in depth, but is not a substitute for this local gate. The replacement implements that gate in one private `accept` function shared by fast paths, custom solvers, generic algebraic proposals, and phase-orbit candidates.

5.2 F2: Ambient assumptions turn an unconditional interface into a conditional one

The recovered `safeSimplify` calls `Simplify` on the completed expression without an explicit assumptions option. Wolfram documents that this inherits `$Assumptions` [13]. In a symbolic host, the final

exact-algebraic equality filter is skipped.

A revealing probe is

```
Block[{$Assumptions = x > 0},
  RadicalDenest`DenestRadicals[Sqrt[x^2]]
]
```

Under that assumption, x is a valid simplification and is cheaper than the radical. Returned without its condition, however, it is not the same function on the originally unspecified domain: at $x = -2$, the principal square root of x^2 is 2, not -2.

This is not a bug in `Simplify`. It is a contract choice that the package must make explicitly. An assumptions-aware interface could return a condition, include assumptions in every certificate, and document the result's domain. A constants-only denester should instead avoid global symbolic cleanup and reset assumptions in its numerical proof operations. The supplied implementation takes the latter route. It does not promise symbolic parameter denesting.

5.3 F3: Positivity of a product does not justify splitting its recorded factors

The visible common-factor helper forms an aggregate common factor F and a residual sum S. It permits a power transformation when the outer exponent is integral or when F or S is a positive real number. The guard can justify splitting $(FS)^p$ into $F^p S^p$ in the appropriate nonzero setting. But the helper reconstructs the common part as powers of the *individual recorded bases*. That is a stronger transformation.

An exact internal-state example is

```
RadicalDenest`Private`combine[
  {{{-1, 2}, {2, 1}}, {{-1, 2}, {3, 1}}}, 1/2
]
```

The factor records represent the two terms $(-1)^2 \cdot 2$ and $(-1)^2 \cdot 3$. Their common factor is $F = (-1)^2 = 1 > 0$, and their residual sum is $S = 5$. The intended value is

$$\sqrt{FS} = \sqrt{5}.$$

Multiplying the common factor's recorded exponent by $1/2$ instead gives

$$(-1)^{2(1/2)}\sqrt{5} = -\sqrt{5}.$$

The aggregate positivity test passes, but the reconstruction has lost a full winding of the argument.

Proposition 5.1 (The missing phase factor)**.** *Suppose $a_j \neq 0$ and $F = \prod_j a_j^{e_j}$, with all powers interpreted by principal logarithms. There is an integer n such that*

$$\sum_j e_j \operatorname{Log} a_j = \operatorname{Log} F + 2\pi i n.$$

Consequently, for a rational outer exponent p,

$$\prod_j a_j^{e_j p} = F^p e^{2\pi i n p}. \tag{5.1}$$

Even $F > 0$ does not force the final factor to be 1.

Proof. Exponentiating the difference of the two logarithmic expressions gives 1, so the difference is an integral multiple of $2\pi i$. Multiplication by p and exponentiation give (5.1). In the example $a_1 = -1, e_1 = 2$, the sum is $2\pi i$, while $\operatorname{Log} F = 0$. Thus $n = 1$ and $p = 1/2$ yields the factor -1. □

Scope of the counterexample. This disproves the helper guard for the displayed internal representation. It does not prove that ordinary evaluated public input reaches that representation. Nor does it prove that `Factorc` returns a wrong numerical value: its visible numerical equality check should reject such a proposal. The appropriate classification is a latent helper defect plus a regression obligation, not a claimed observed public wrong answer.

A narrow repair preserves the aggregate as F^p instead of splitting it again. A stronger local rule demands branch-safe conditions for every individual power merge. The replacement avoids this handwritten factor representation entirely: `Factor` may propose an expression, but the same exact acceptance gate decides whether it may replace the original numerical target.

5.4 A branch guard that should not be condemned

The visible condition permitting $(a^b)^c \mapsto a^{bc}$ when c is integral or when b is real with $-1 < b \le 1$ has a sound nonzero complex rationale. If $a = \rho e^{i\theta}$ with $-\pi < \theta \le \pi$, the latter interval for b keeps $b\theta$ inside the principal argument interval. Hence $\operatorname{Log}(a^b) = b \operatorname{Log} a$ and the merge is valid. The endpoint $b = -1$ is rightly excluded: $a = -1$ reaches the opposite side of the cut. Zero and undefined negative powers still need their ordinary domain checks.

This is an example of why a review should distinguish a valid *single-power* guard from an invalid inference about an entire factor list. They are related, but not the same proof obligation.

6 Resource, configuration, and traversal behavior

6.1 F4: A local time limit is not a whole-call budget

The visible wrapper performs algebraicity checks, numerical evaluation, factorization, radical normalization, marker manipulation, callbacks, and cleanup outside any single encompassing `TimeBudget` region. Passing that option to `DenestCore` cannot bound the surrounding work. List dispatch and recursive re-entry also allow multiple local budgets rather than one shared request budget [1].

For m numerical islands, a structural accounting is

$$T_{\text{request}} = T_{\text{classification}} + T_{\text{preprocessing}} + \sum_{j=1}^{m} \left(T_{\text{search},j} + T_{\text{verification},j} \right) + T_{\text{reconstruction}}.$$

A bound on each search term alone is not a bound on this sum. Even within the equality predicate, algebraicity and the numerical prefilter occur before the timed exact operations. Multiple candidate certifications multiply the nominal certification allowance.

A `TimeBudget -> 0` callback-counter probe is included in the original-source probes. It asks whether the callback is entered at all, rather than relying on unreliable elapsed-time comparisons. Its expected effect follows from the visible placement of the callback; it remains a native test to run against the complete source.

Repair and its honest limits. The replacement wraps traversal, generation, proof, and cost computation in one kernel resource region. A single trial counter and deadline are shared across lists and passes. Stage limits use the lesser of the per-stage allowance and remaining deadline. Timeout and memory-limit outcomes roll back to the original expression, avoiding a half-reconstructed host.

Wolfram's resource primitives still have documented limitations. `TimeConstrained` is interrupt-based, has finite granularity, and is not a hard operating-system wall-clock bound; protected evaluation and external work can defeat an overly literal promise. `MemoryConstrained` is a kernel allocation control, not a container's resident-set limit [14, 15]. Argument evaluation before function entry, option resolution, and lightweight final reporting remain outside the engine budget. The independent public factoring and rationalization helpers have separate proposal and proof allowances, not the entry-point budget.

6.2 F5: Effective option defaults disappear during delegation

There are two distinct issues. First, the visible alias `Strad` delegates only supplied option rules to `DenestRadicals`; it does not resolve its own defaults. Therefore setting a default on the alias does not by itself inject that setting into the delegated call. Second, the wrapper builds core options by filtering the *explicit rules*, not all effective wrapper option values. A changed wrapper default need not agree with the core's independent default.

The distinction matters for standard usage such as

```
SetOptions[RadicalDenest‘Strad, "TimeBudget" -> 0];
RadicalDenest‘Strad[expression]
```

The alias's declared option table is not automatically consulted merely because it exists. Options are part of executable configuration flow, not just documentation [16].

The replacement resolves the current defaults of the actual entry point, gives the first explicit rule precedence, normalizes option lists, evaluates delayed values into one configuration, validates that configuration, and passes it as private call state. Recursive workers do not reconstruct defaults. This also prevents accidental resetting of a remaining budget.

6.3 F8: The factor switch has a narrower effect than its name suggests

The outer `Factor` option controls `safeFactorAll`. However, the recovered `normalizeRadicand` invokes `Factorc` internally regardless of that switch. Thus disabling the outer factor pass does not disable all factoring work. This may be an intentional distinction between preliminary factoring and mandatory normalization, but it is not apparent from the option name.

The issue is both semantic and operational: a user trying to bypass expensive factoring cannot infer that `Factor -> False` accomplishes it. The API should either document separate phases or provide independent switches. In the replacement, `"Factor" -> False` suppresses its explicit `Factor` proposal. It does not claim to control proprietary internal factoring performed inside `Simplify`, `MinimalPolynomial`, or other kernel algorithms.

6.4 F9: A remaining opaque root cancels unrelated successful conversions

The visible conversion path attempts `ToRadicals` on the whole expression and then restores the original expression if any `Root` or `AlgebraicNumber` remains. A host containing two different algebraic

constants can therefore lose a successful conversion of one merely because the other was not converted.

The better acceptance unit is the individual numerical island or root occurrence. A mixed representation with fewer opaque roots can be retained when it satisfies the chosen cost and equality policy. This is a coverage improvement, not evidence that `ToRadicals` itself is erroneous. The replacement no longer applies that all-or-nothing rollback.

6.5 F10: Markers add proof obligations beyond algebra

A marker-based implementation needs invariants about arity, marker nesting, dependency ordering, rule construction, termination, and complete removal of internal symbols. A finite iteration cap establishes a bound on one loop; it does not establish that the resulting object is a fully solved expression. An unresolved marker in a symbolic host is particularly concerning when the whole-host numerical certification is unavailable.

No infinite loop or marker leak was established for the recovered corrected source. The missing helper/backend bodies and absent native run prevent that conclusion. The recommendation is architectural and test-driven: assert that no private marker or failure token escapes; distinguish exhaustion of a dependency-resolution cap from successful completion; and preserve the input on unresolved internal state.

The replacement removes this intermediate language. Bottom-up traversal and bounded passes work directly with ordinary expressions. This sacrifices some potentially useful grouping heuristics, but shrinks the amount of state whose correctness must be maintained.

6.6 Conservative handling of inexact inputs is not a mathematical bug

The recovered wrapper declines inexact expression trees. This can miss exact islands inside a mixed tree, but it has a legitimate purpose: changing an exact subexpression may alter when surrounding floating-point arithmetic rounds. The replacement retains the conservative policy and documents it. Supporting mixed trees would require a different numerical-error contract, not simply deleting the guard.

7 Equality certification and progress metrics

7.1 F6: Unknown is neither unequal nor proved equal

An equality service has three useful outcomes:

$$\text{Equal}, \qquad \text{Different}, \qquad \text{Unknown}.$$

Resource exhaustion, unsupported syntax, and an unevaluated exact operation belong in the third category. A Boolean acceptance predicate may safely map unknown to false, but a diagnostic layer must not report that as an established inequality.

The recovered predicate uses a numerical approximation to reject some candidates before exact arithmetic. This cannot create a false accepted equality by itself, since successful candidates still face an exact check. It can nevertheless reject a genuine identity if the approximate residual is unreliable, and it consumes work outside the timed exact step. A fixed absolute threshold is not an error enclosure. Wolfram's discussion of `$MaxExtraPrecision` explicitly warns about lost requested precision and hidden zeros [18].

No native false-rejection example was executed here. The finding is a reliability and evidence problem, not an asserted observed miscalculation. Numerical evaluation is useful for ordering candidates or obtaining rigorous enclosures. It should not be treated as a proof of non-equality without a certified exclusion of zero.

The replacement removes this numerical prefilter. It accepts only a literal zero obtained from exact reduction of the difference. A returned explicit nonzero canonical algebraic object can be classified as different; other outputs are unknown. A timeout is not silently upgraded to a theorem about equality or non-denestability.

7.2 Why not always call a second exact oracle?

`PossibleZeroQ` with the exact-algebraic method is a legitimate alternative on its documented domain [12]. Retaining it can improve coverage when an initial conversion pathway fails. The small reference implementation deliberately uses a single acceptance mechanism instead, reducing the trusted surface and making unknown outcomes visible. This is a simplicity-versus-coverage decision, not a claim that the documented fallback is unsound.

A production implementation may use several exact proof procedures, but should attach their domain assumptions and result types. In particular, the expression `RootReduce[d] === 0` evaluates to false even when the inner call remains unevaluated. That is fine as a conservative *acceptance failure*; it is insufficient as a certificate that $d \neq 0$.

7.3 F7: A score cannot replace an admissibility check

The recovered usage text advertises a four-component cost, while the visible definition includes an additional atom-count component. More substantively, it counts certain syntactic heads rather than defining all admissible numerical presentations. Neither issue proves wrong numerical equality, but both weaken the meaning of "strictly cheaper."

A score also needs to account for representation growth. Leaf count treats a huge integer as one leaf. A root count does not charge for a very large root index. A depth-only objective can favor a very long sum of shallow radicals. These observations support a small integer cost tuple, explicit syntax restrictions, and separate byte/leaf/index limits rather than an opaque scalar score.

The replacement's bit-cost term and size gates are improvements, not a perfect readability model. Exponent bit length contributes to the cost, but mathematical depth remains syntactic. Algebraic degree is not included as a cheap global component because obtaining it can be the expensive part of the problem itself.

8 The multiplier–polynomial-GCD mechanism

This section analyzes the mathematical mechanism identified in the recovered package description and earlier review material. It is not a line-by-line reconstruction of the unavailable current core. The same mechanism is implemented independently in the supplied reference code.

8.1 The construction and its coefficient field

Let β be the *original selected target value*. Choose $r \geq 2$ and put $R = \beta^r$. For a nonzero algebraic multiplier m, form

$$\alpha = (mR)^{1/r}, \qquad p_m(X) = \mathrm{MinPoly}_{\mathbb{Q}}(\alpha),$$

and compute

$$g_m(X) = \gcd\big(p_m(X), X^r - mR\big) \tag{8.1}$$

over a field K containing the algebraic coefficients used to represent mR. The coefficient field matters: treating displayed radicals as independent formal variables changes the computation. Wolfram's `Extension -> Automatic` is documented to extend the field by algebraic numbers occurring in the polynomial coefficients [17].

Proposition 8.1 (What the GCD certifies)**.** *Assume characteristic zero and exact valid inputs. The polynomial g_m is nonconstant. Every root a of g_m satisfies $a^r = mR$. If $\deg g_m < r$, at least one scaled root obeys a polynomial equation of smaller degree than the binomial.*

Proof. The number α is a common root of p_m and $X^r - mR$ in an algebraic closure of K. If their GCD were constant, Bézout's identity over $K[X]$ would give a linear combination equal to 1. Evaluation at α would give $0 = 1$. Thus the GCD is nonconstant. Since it divides the binomial, each of its roots satisfies that binomial. The degree comparison is immediate. □

The conclusion concerns an *equation*, not automatically a radical presentation. Coefficients of a small-degree polynomial may already be deeply nested, and a solver can produce an expression that is longer or more nested than the input. Consequently a proper GCD is a candidate-generation success, not an acceptance certificate or a progress measure.

8.2 The complete phase orbit

Theorem 8.2 (Branch recovery from any one GCD root)**.** *Suppose $m \neq 0$, $R \neq 0$, $r \geq 2$, and $a^r = mR$. Let $\mu = m^{1/r}$ be the principal root and $\zeta_r = e^{2\pi i/r}$. Then*

$$\left\{ \frac{a}{\mu} \zeta_r^k : 0 \leq k < r \right\} \tag{8.2}$$

is exactly the set of roots of $X^r = R$. In particular it contains the original β whenever $\beta^r = R$.

Proof. Because $\mu^r = m$ and $\mu \neq 0$, $(a/\mu)^r = R$. Multiplication by ζ_r^k does not change the rth power. The r resulting values are distinct since $a/\mu \neq 0$ and the roots of unity are distinct. A degree-r polynomial has at most r roots, so the set is complete. □

No step assumes $(mR)^{1/r} = m^{1/r} R^{1/r}$. Indeed that identity is unsafe for principal complex powers. If $R = 0$, the only value is zero and should be handled separately; it does not require a distinct r-element orbit.

For implementation, ζ_r^k can be represented by $(-1)^{2k/r}$, which stays within the rational-power language. One root of the GCD already provides complete *value* coverage. Other roots may produce different and useful *presentations*; they should not be justified as necessary merely to recover the desired branch.

The original target must remain available. For example, $\beta = -1/2 + i\sqrt{3}/2$ satisfies $\beta^3 = 1$, while the principal cube root of 1 is 1. Replacing the target by $R^{1/r}$ would erase its embedding. Checking the final candidate against β avoids that loss.

8.3 A complete worked multiplier example

Take

$$R = 5 + 2\sqrt{6}, \qquad \beta = \sqrt{R} = \sqrt{2} + \sqrt{3}.$$

For $m = 1$, the minimal polynomial is

$$p_1(X) = X^4 - 10X^2 + 1.$$

Over $K = \mathbb{Q}(\sqrt{6})$, its GCD with $X^2 - R$ is the entire binomial. That trial provides no smaller-degree equation.

For $m = 2$, however,

$$(2 + \sqrt{6})^2 = 2R, \qquad p_2(X) = X^2 - 4X - 2,$$

and

$$\gcd(p_2(X), X^2 - 2R) = X - 2 - \sqrt{6}.$$

Thus the scaled root $a = 2 + \sqrt{6}$ gives

$$\frac{a}{\sqrt{2}} = \sqrt{2} + \sqrt{3}.$$

The positive value is selected by equality to the original principal square root. The accompanying script verifies the two GCDs, minimal polynomials, and reconstruction exactly.

The discriminant $\operatorname{disc}(p_1) = 147456 = 2^{14}3^2$ makes the primes 2 and 3 plausible multiplier seeds. This illustrates a heuristic motivation, not a completeness theorem. Failure to find a multiplier among such primes does not prove that no useful multiplier exists.

8.4 Why degree and resource gates must be described precisely

A `MaxDegree` check after `MinimalPolynomial` prevents expensive downstream GCD or solving work. It does not prevent the minimal-polynomial calculation itself from visiting a large extension. The replacement labels its degree cap as a post-computation admission check and relies on the encompassing resource controls during construction.

For a compact expression involving several independent radicals, field degree can be much larger than syntax size. A formula of modest depth can still generate an exponentially large compositum. This representation-sensitive difficulty is already central in the general literature; a local heuristic should not be assigned a polynomial-time guarantee merely because its individual polynomial operations are standard [5, 8, 9].

9 Restricted fast paths with short certificates

The current source prefix announces a later quadratic helper, but its complete body was not retrieved. The implementation below therefore does not claim that elementary norm methods are new to the repository. They are derived here to make the independently written fast path and its exact tests self-contained.

9.1 Direct quadratic denesting

Let $a, b, c \in \mathbb{Q}$, $c > 0$ nonsquare, $b \neq 0$, and $a + b\sqrt{c} > 0$. Seek

$$\sqrt{a + b\sqrt{c}} = \sqrt{u} + \varepsilon\sqrt{v}, \qquad u, v \in \mathbb{Q}_{\geq 0}, \quad \varepsilon \in \{\pm 1\}.$$

Squaring requires

$$u + v = a, \qquad uv = \frac{b^2 c}{4}.$$

Therefore u, v are the roots of $X^2 - aX + b^2c/4$. Put $D = a^2 - b^2c$. If $d = \sqrt{D}$ is rational, then

$$\sqrt{a + b\sqrt{c}} = \sqrt{\frac{a+d}{2}} + \operatorname{sgn}(b)\sqrt{\frac{a-d}{2}}. \tag{9.1}$$

The positive-radicand hypotheses and $D \geq 0$ imply $a > 0$ and $a \geq d$. Hence both inner rational radicands are nonnegative. The square of the right side is the original radicand, and its sign is positive; this selects the principal square root. Direct and indirect versions of this local theory are discussed by BFHT and Gkioulekas [5, 6].

For $a = 5, b = 2, c = 6$, one obtains $d = 1$, so $u = 3, v = 2$. For $a = 3, b = -2, c = 2$, the sign gives $\sqrt{2} - 1$, not $1 - \sqrt{2}$. The latter example is a necessary regression case because the algebraic square identity alone cannot distinguish those signs.

9.2 Indirect fourth-root denesting

A nonsquare norm does not rule out a depth-one expression that uses fourth roots. Suppose

$$h = \sqrt{b^2 - a^2/c} \in \mathbb{Q}_{\geq 0}, \qquad b > 0,$$

in the genuine indirect positive-real case. Then

$$\sqrt{a + b\sqrt{c}} = c^{1/4}\left(\sqrt{\frac{b+h}{2}} + \operatorname{sgn}(a)\sqrt{\frac{b-h}{2}}\right). \tag{9.2}$$

Indeed the square of the parenthesis is

$$b + \operatorname{sgn}(a)\sqrt{b^2 - h^2} = b + \frac{a}{\sqrt{c}},$$

so multiplication by $\sqrt{c}$ gives $a + b\sqrt{c}$. Positivity of the selected right side completes the verification. The case $a = 0$ is elementary and can be handled directly or by candidate verification.

The example

$$\sqrt{4 + 3\sqrt{2}} = 2^{1/4}(1 + \sqrt{2})$$

has $D = -2$ and $h = 1$. The code generates sign variants and subjects each to the same exact local acceptance gate. This permits safe complex candidates outside the displayed positive-real derivation without silently extending that derivation's hypotheses.

9.3 Restricted completeness is not a global contract

The classical local theory distinguishes square-root-only and fourth-root alternatives under precise hypotheses [5, 6]. A recognizer for the visible two-term syntax is not automatically a complete implementation of that theorem: equivalent radicands may need normalization, recognition may fail, or resources may expire.

Nor is an unchanged output a proof of impossibility. A particularly instructive distinction is

$$\sqrt{2+\sqrt{2}} = \zeta_{16} + \zeta_{16}^{-1}.$$

A restriction to real radicals is different from permitting complex roots of unity. The package does not return theorem-based non-denestability certificates for either class, and the replacement always reports that its search is incomplete.

10 The proposed implementation and its invariants

10.1 A smaller trusted transformation boundary

The file `code/StradImproved.wl` uses a separate `RadicalDenestImproved‘` context. This allows side-by-side testing without overwriting the original package. Its high-level sequence is

```
resolve and validate options once
open one shared resource region
walk permitted pure expression heads
  identify an explicit numerical island
  obtain candidate expressions
  reject inadmissible or oversized candidates
  require a strictly lower declared cost
  require exact equality to the original island
  record the accepted replacement
rebuild the host by congruence
return a complete result, or roll back on interruption
```

The generator is allowed to be incomplete. It is not allowed to decide that its own output is correct merely because it found a smaller equation or a promising numerical approximation.

10.2 Candidate sources actually implemented

The reference includes the direct/indirect quadratic proposals, exact algebraic reduction, low-degree radical conversion, optional built-in factoring, numerical-island simplification with explicit assumptions reset, user-solver proposals, and a finite multiplier/GCD/phase search. The default GCD solver explicitly handles linear and quadratic equations. An option permits cubic or quartic `Solve` proposals; these still undergo syntax, size, cost, and equality checks.

Automatic multipliers include 1, a bounded positive-integer range, and visible complementary rational-power seeds. Explicit multiplier lists are allowed; zero, inexact, unsupported, or uncertifiably nonzero entries are skipped. Structural duplicates are removed. The reference does not reconstruct the unavailable source's full discriminant-based or history-based multiplier strategy, and makes no claim of equal search coverage.

10.3 Conditional soundness of the design

Theorem 10.1 (Soundness of accepted reference-design outputs)**.** *Assume ordinary pure scalar Wolfram semantics for the supported host operations; correct exact algebraic reduction on the admitted numerical inputs; and faithful execution of the stated acceptance and rollback logic. Then every completed accepted transformation preserves the selected value of the input wherever it was defined. This assertion does not depend on the mathematical reliability of the candidate generator.*

Proof. Each committed numerical replacement has a literal-zero exact difference certificate against its original island, so its selected value is equal. Proposition 3.1 lifts these equalities through the supported host. Finite passes compose equalities. If the outer resource region is interrupted, the original input is returned instead of the incomplete reconstruction. Thus both successful completion and the specified rollback preserve the original selected value. □

This is a proof of the *design under stated implementation assumptions*, not a formal verification of every Wolfram line. Native parsing, option semantics, exception handling, and regression behavior still need the supplied kernel tests. It would be circular to cite this theorem as evidence that an unexecuted implementation cannot contain a programming error.

Proposition 10.2 (Progress and bounded traversal)**.** *For successfully committed steps, the fixed cost tuple in* (3.2) *strictly decreases. Therefore there is no infinite sequence of committed decreases. Explicit finite pass and trial limits additionally bound the reference engine's search loops, subject to termination or interruption of individual kernel operations.*

Proof. Lexicographic order on a fixed finite product of nonnegative integers is well-founded. To see this inductively, the first coordinate can decrease only finitely often; after its final decrease it is fixed, and the remaining coordinates form a smaller well-founded lexicographic product. The acceptance gate enforces strict descent. Separately, the pass counter and the shared trial counter have finite configured maxima, and each phase orbit has bounded cardinality r. □

This excludes endless *accepted progress*, but does not make a large rejected proposal inexpensive. That is why per-stage limits and the outer resource region remain necessary.

10.4 Reporting and rollback

`DenestReport` returns the result, a status, trial count, stage-failure count, unknown-certificate count, degree skips, accepted-local-step count, and bounded local equality records. Its main statuses are `Improved`, `Unchanged`, `Timeout`, `MemoryLimit`, `Disabled`, and `InvalidOptions`. An unchanged result still conflates some benign cases, including unsupported syntax and no useful candidate; the counters and source-access notes prevent it from being called a non-denestability proof.

The equality records identify before/after numerical expressions and an exact-zero residual. They are useful audit records, but not portable proof objects independent of Wolfram. They can include intermediate steps discarded by a subsequent whole-call rollback. The maximum retained record count is configurable.

The conservative rollback policy returns the original expression on a whole-call time or memory interruption. Retaining the best fully committed host would be a reasonable enhancement, but requires a transactional checkpoint of the *complete* host. A partly updated list of local candidates is not such a checkpoint.

10.5 Defaults and compatibility decisions

Option	Default	Meaning
`"TimeBudget"`	60	Shared engine allowance; zero disables transformation.
`"StageTime"`, `"CertifyTime"`	5, 5	Per-stage upper allowances, reduced by remaining deadline.
`"MaxTrials"`	64	Shared count of expensive multiplier trials.
`"MultiplierCap"`	32	Automatic positive-integer seed range; not a universal multiplier bound.
`"AllLevels"`, `"MaxPasses"`	False, 3	Enable bounded bottom-up passes, not arbitrary fixed-point rewriting.
`"MaxDegree"`	64	Post-computation minimal-polynomial admission limit.
`"MaxRootIndex"`	24	Limit on the GCD phase-search index.
`"MaxSolveDegree"`	2	Linear/quadratic by default; at most four when increased.
`"MaxLeafCount"`, `"MaxBytes"`	4096, 4 MiB	Expression-size gates.
`"MemoryBudget"`	256 MiB	Kernel memory constraint for the engine.
`"MaxRecords"`	128	Maximum local equality records retained.

The familiar `Strad`, `DenestRadicals`, and `DenestCore` names and the Boolean all-levels shorthand are present, but this is not a drop-in equivalence claim. Symbolic `Factorc` behavior is deliberately narrowed; arbitrary heads and inexact trees are not traversed; the score and default search policy differ. The usage guide lists these differences before giving migration examples.

10.6 Remaining weaknesses of the reference implementation

The replacement itself needs criticism. It has no native execution evidence yet. Its recursive tree walk is not a shared-subexpression graph, so repeated islands can be recomputed. It uses structural rather than algebraic duplicate detection for seeds. Repeated `RootReduce` calls can dominate runtime. Its minimal-polynomial degree limit is late. Built-in simplification can emit benign messages that the conservative `Check` policy treats as failure. The top-level outcome taxonomy could distinguish unsupported input and every particular resource gate more precisely.

The `Root` conversion and bounded-pass policies can leave denestings undiscovered. A purely local search misses transformations that temporarily increase the cost or require a simultaneous regrouping of several terms. An arbitrary callback can still have side effects or attempt to defeat time constraints; local mathematical validation is not process isolation. These are concrete limitations, not reasons to weaken the equality boundary.

11 Verification, test coverage, and the release gate

11.1 What the executed checks establish

The delivered script ran with Python 3.13.5 and SymPy 1.14.0. All **165 checks passed**: 156 exact mathematical checks, four lexical-balance checks, and five static structural checks. The mathematical portion includes a generated family of direct-norm examples, selected branches, both worked polynomial GCDs, minimal polynomials and discriminant, conjugate separation, the aggregate-factor phase counterexample, denominator rationalization, cubic norm/trace identities, and a Honsbeek quartic certificate.

No floating-point agreement is used as an equality certificate in those checks. Squared identities have separate exact sign facts where branch selection is claimed. The generated family verifies the algebra behind the fast path, not whether Wolfram's pattern matcher recognizes all those inputs.

The lexical check balances ordinary delimiters while respecting strings and nested Wolfram comments. It is not a Wolfram parser. The static checks confirm selected architectural features in the delivered text; they do not prove absence of other bugs. The complete per-check names and results are included in `evidence/verification.json` and `evidence/verification.txt`.

11.2 The native suite that still must run

Thirty-one `VerificationTest` cases are supplied. They cover direct and indirect denesting values and depths; a Ramanujan cubic case with multiplier nine; a small positive branch; negative square-root and odd-root conventions; incorrect, failed, and approximate solver results inside a symbolic host; ambient assumptions; opaque heads; exact syntax recognition; conjugate inequality; zero and invalid budgets; option-list handling; changed alias defaults; a shared list trial budget; simple idempotence; inexact-tree policy; rationalization; symbolic factoring refusal; invalid multiplier seeds; and bounded equality records.

The portable runner loads the package relative to its own path and calls `TestReport`. There is no claim that this suite has passed on Wolfram: it has not been run here. Some cases validate correctness even when the search returns unchanged; separate depth assertions test actual simplification on the elementary examples. This separation avoids confusing safety with search power.

11.3 Why independent algebra does not replace kernel testing

A correct polynomial identity can coexist with a broken Wolfram option pattern, an unintended held evaluation, an incorrectly shaped association, or a resource-abort edge case. Conversely, a passing handful of kernel examples does not prove a field-theoretic completeness theorem. Both layers are required.

Before adopting the replacement, run the native suite in a clean kernel, record `$Version`, and add the exact Git commit and file hash of the comparison target. Then run the original-source probes, which target the recovered wrapper rather than assuming the unseen core's behavior. Any mismatch with the predicted probes should refine the finding; it should not be concealed to preserve the narrative.

11.4 A useful differential benchmark protocol

For each test input, retain the original expression, branch convention, admissible output language, software versions, resource settings, output, exact equality result, depth, size, and outcome status. Compare values rather than a single preferred printed expression. Keep generation time and certification time separate. Record warm-cache and clean-kernel results separately if caching is introduced.

A representative corpus should include direct squares, fourth-root denestings, complex phase examples, mixed square/cube identities, large rational coefficients, close cancellation, multiple numerical islands, symbolic hosts, and cases known to require a different output model. A list of beautiful identities alone is not a reliable performance or correctness benchmark.

12 Further algorithmic improvements grounded in the literature

12.1 Dependency-aware simplification before repeated local search

BFHT explicitly exhibit global cancellation among individually troublesome radicals [5]. Let $u = \sqrt{1+\sqrt{3}} > 0$. Then

$$\sqrt{3+3\sqrt{3}} = \sqrt{3}\,u, \qquad \sqrt{10+6\sqrt{3}} = (1+\sqrt{3})u,$$

so

$$\sqrt{1+\sqrt{3}} + \sqrt{3+3\sqrt{3}} - \sqrt{10+6\sqrt{3}} = 0.$$

This suggests a graph of algebraic dependencies, not just a queue of isolated outer roots. A practical next step is to cache exact normal forms of repeated islands and use certified ratios or products to group dependent terms. The reference's whole numerical-island `RootReduce` proposal can catch some cancellation, but it is not an efficient specialized dependency algorithm.

Cache entries should store their supported domain, selected embedding, and failure status. An unknown result obtained under a short budget must not become a permanent negative answer under a larger budget. Context-sensitive symbolic assumptions are another reason to keep the first cache numerical and per call.

12.2 A cubic-in-quadratic module

For real cube-root output constrained to $\mathbb{Q}(\sqrt{p})$, write

$$(A + B\sqrt{p})^3 = a + b\sqrt{p}, \qquad a, b, p, A, B \in \mathbb{Q},$$

with $p > 0$ nonsquare and $b \neq 0$. Taking norms gives

$$a^2 - b^2 p = t^3, \qquad t = A^2 - pB^2.$$

Putting $x = 2A$ yields

$$x^3 - 3tx - 2a = 0, \qquad A = x/2, \qquad B = \frac{b}{x^2 - t}. \tag{12.1}$$

The sufficiency of this reconstruction follows from the polynomial identity

$$(x^3 - 3tx)^2 - 4t^3 = (x^2 - t)^2(x^2 - 4t). \tag{12.2}$$

Under the stated equations, its left side is $4b^2p$, so the denominator in (12.1) is nonzero and $pB^2 = A^2 - t$. Substitution then verifies the rational and irrational parts of the cube. This gives a small complete module for a *specified field-output problem*, not unrestricted cubic denesting.

A cube norm alone is insufficient: $a = b = 1, p = 2$ gives norm -1, but $x^3 + 3x - 2$ has no rational root. Rational polynomial factorization, rather than an unrestricted integer-divisor search, is a natural implementation route. Negative real radicands still require conversion between the real cube root and the package's principal complex-power convention. The module is a proposed extension, not present in the supplied Wolfram reference.

12.3 Honsbeek's mixed-index criterion as a plugin

For nonzero rational α, β with $q = \beta/\alpha$ not a rational cube, Honsbeek's Theorem 104 relates denesting of $\sqrt{\sqrt[3]{\alpha} + \sqrt[3]{\beta}}$ in the stated radical-closure model to a rational root of

$$F_q(s) = s^4 + 4s^3 + 8qs - 4q.$$

The formula on printed page 63 uses compatible roots and an appropriate sign [8]. Its reconstruction can be checked by the compact polynomial certificate

$$\left(-\frac{s^2}{2} + su + u^2\right)^2 - (u^3 - s^3)(1 + u) = \frac{F_{u^3}(s)}{4}. \tag{12.3}$$

This is a good candidate-generator module because both its recognition conditions and algebraic residual are explicit. Branch selection, the noncube assumption, and nonvanishing denominators must still be checked. Merely applying the displayed formula to unrelated syntax would lose the theorem's hypotheses.

The reference does not implement this module. The exact certificate and two rational-root examples are included in the Python checks so that a future implementation has a small algebraic regression foundation.

12.4 Field-aware multiplier generation and phase reduction

The multiplier search can exploit known field structure more directly than raw integer enumeration. Useful experiments include deriving candidates from norm factorizations, tracking which extension is represented by the current coefficients, and choosing a basis that accounts for multiplicative relations before coefficient comparison. Honsbeek's analysis of radical extensions and roots of unity explains why such information is structural rather than a matter of notation [8].

The phase-orbit theorem also suggests an optimization: certify one root of the GCD and enumerate its orbit first. Search additional roots only for a better presentation. Candidate equality normal forms can eliminate duplicate work, but phase selection must remain tied to the original target, not to a nearby numerical value or a common minimal polynomial.

Any discriminant-based multiplier generator should state what its cutoff means. A bound on multiplier magnitude, a bound on the number of seeds, and a bound on the algebraic degree of a multiplier are different controls. Progress history should never turn "not tried within this budget" into "mathematically excluded."

12.5 Portable certificates and stronger isolation

The present equality records trust Wolfram's exact algebraic arithmetic. A more portable certificate would contain a defining polynomial, exact coefficient relations, nonzero denominator evidence, and a root-isolating interval or complex region identifying the selected value. For restricted formulas, a polynomial remainder plus exact sign information is often much smaller than a full field description.

For strict time and memory guarantees, execute candidate generators in separate worker processes controlled by an operating-system supervisor. Accept only serialized expressions in a restricted grammar, then check them in a separate trusted verifier. That design addresses both uninterruptible work and the difference between an untrusted mathematical answer and hostile executable code. It is an architectural extension, not a property provided by `TimeConstrained` alone.

13 Recommended disposition

The recovered corrected source has the right core instinct: preserve the original selected value and insist on exact evidence for numerical changes. The most important remaining repair is to apply that instinct at *every* public replacement boundary. Symbolic hosts cannot be protected by a test that runs only for wholly numerical inputs.

The immediate integration sequence is to recover and pin the complete current source; run the supplied wrapper probes and native regression suite; repair local callback acceptance and assumptions scope; resolve options once and share budgets; and only then compare search coverage and performance. The helper-level winding example belongs in the regression corpus even if public reachability is disproved, because it documents why the aggregate guard is insufficient in isolation.

The independently written implementation supplies a concrete basis for that work. It reduces the state machinery, makes its expression model and failure outcomes visible, and keeps mathematical proof separate from heuristic discovery. It is not presented as a finished production replacement, a complete review of unavailable source, or a proof that the original repository's unseen backend is defective.

A How to reproduce the delivered evidence

From the archive root, run

```
python tests/verify_math.py
wolframscript -file tests/run_tests.wls
pdflatex -interaction=nonstopmode -halt-on-error review.tex
pdflatex -interaction=nonstopmode -halt-on-error review.tex
```

Only the Python and LaTeX commands were executed in this environment. The native Wolfram command is the next validation step. The exact-check output identifies every assertion and the software versions; it does not contain invented Wolfram output.

Use fully qualified package names when comparing implementations:

```
Get["/path/to/StradFixed.wl"];
Get["/path/to/StradImproved.wl"];
old = RadicalDenest‘Strad[expr];
new = RadicalDenestImproved‘Strad[expr];
RadicalDenestImproved‘EqualityStatus[old, expr]
```

```
RadicalDenestImproved`EqualityStatus[new, expr]
```

The independent implementation deliberately does not overwrite the original definitions. A literal unchanged result counts as safe but not as a successful denesting. The case `"MaxTrials" -> 0` disables only generic multiplier trials; `"TimeBudget" -> 0` is the explicit no-transformation mode.

B Complete proposed Wolfram implementation

The following is the exact proposed implementation included in the archive. It was lexically checked, not parsed or executed by a Wolfram kernel. Its inline comments and the contract in Section 10 are part of the deliverable.

```
(* ::Package:: *)
(* Independently written reference implementation, 2026-09-05.
   NOT a source-complete patch of StradFixed.wl. See SOURCE_ACCESS.md.
   Native Wolfram Language execution was unavailable during this review.
   Scope: exact scalar arithmetic; principal complex Power; fail-closed
   local certification. Custom solvers are proposals, not proof oracles.
   This is not a sandbox for hostile Wolfram Language programs. *)

BeginPackage["RadicalDenestImproved`"];
Strad::usage = "Strad[e, options] attempts certified radical simplification. Strad[e, True]
    enables repeated bottom-up passes.";
DenestRadicals::usage = "DenestRadicals[e, options] returns a certified improvement or the
    original evaluated expression.";
DenestCore::usage = "DenestCore[e, options] restricts the entry point to an explicit exact
    algebraic expression.";
DenestReport::usage = "DenestReport[e, options] returns the value, outcome, counters, and bounded
    local equality records.";
ExactAlgebraicQ::usage = "ExactAlgebraicQ[e] conservatively recognizes the supported explicit
    algebraic grammar; it is not a universal algebraicity decision procedure.";
EqualityStatus::usage = "EqualityStatus[a,b,t] returns Equal, Different, or Unknown as a string,
    using exact RootReduce under a time limit.";
CertifiedEqualQ::usage = "CertifiedEqualQ[a,b,t] is True only when EqualityStatus[a,b,t] is Equal
    (default t=5).";
RadicalDepth::usage = "RadicalDepth[e] measures rational-Power depth. Root objects are separately
    penalized by RadicalCost.";
RadicalCost::usage = "RadicalCost[e] is {opaque algebraic objects, depth, rational-power nodes,
    integer bit cost, leaf count}.";
RationalizeDenominator::usage = "RationalizeDenominator[e,t] attempts an exactly certified
    rational-denominator form of a supported algebraic number.";
Factorc::usage = "Factorc[e,t] is a certified numeric Factor proposal, not the original symbolic
    factoring interface.";

Options[DenestRadicals] = {
  "AllLevels" -> False, "Verbose" -> False, "Multipliers" -> Automatic,
  "MaxTrials" -> 64, "TimeBudget" -> 60, "MultiplierCap" -> 32,
  "CertifyTime" -> 5, "StageTime" -> 5, "Solver" -> Automatic,
  "Factor" -> True, "MaxPasses" -> 3, "MaxDegree" -> 64,
  "MaxRootIndex" -> 24, "MaxSolveDegree" -> 2,
  "MaxLeafCount" -> 4096, "MaxBytes" -> 4194304,
  "MemoryBudget" -> 268435456, "MaxRecords" -> 128
};
Options[Strad] = Options[DenestRadicals];
Options[DenestCore] = Options[DenestRadicals];
Options[DenestReport] = Options[DenestRadicals];
```

```
34
35 Begin["`Private`"];
36 rationalQ[e_] := MatchQ[e, _Integer | _Rational];
37 finiteNonnegativeQ[e_] := NumberQ[e] && TrueQ[Im[e] == 0] && TrueQ[e >= 0];
38
39 (* This grammar deliberately excludes trigonometric/special-function disguises,
40    arbitrary numeric heads, inexact constants, and symbolic parameters. *)
41 ExactAlgebraicQ[e_] := Which[
42   rationalQ[e], True,
43   Head[e] === Complex, rationalQ[Re[e]] && rationalQ[Im[e]],
44   AtomQ[e], False,
45   MemberQ[{Plus, Times}, Head[e]], AllTrue[List @@ e, ExactAlgebraicQ],
46   Head[e] === Power && Length[e] == 2,
47     rationalQ[e[[2]]] && ExactAlgebraicQ[e[[1]]],
48   MemberQ[{Root, AlgebraicNumber}, Head[e]],
49     TrueQ[NumericQ[e] && Precision[e] === Infinity &&
50       Quiet[Check[Element[e, Algebraics], False]]],
51   True, False
52 ];
53
54 RadicalDepth[e_] := Which[
55   AtomQ[e], 0,
56   MemberQ[{Root, AlgebraicNumber}, Head[e]], 0,
57   Head[e] === Power && MatchQ[e[[2]], _Rational], 1 + RadicalDepth[e[[1]]],
58   True, Max[Prepend[RadicalDepth /@ (List @@ e), 0]]
59 ];
60 bitCost[n_Integer] := IntegerLength[Abs[n], 2];
61 bitCost[q_Rational] := bitCost[Numerator[q]] + bitCost[Denominator[q]];
62 bitCost[z_Complex] := bitCost[Re[z]] + bitCost[Im[z]];
63 bitCost[_] := 0;
64 RadicalCost[e_] := {
65   Count[e, _Root | _AlgebraicNumber, {0, Infinity}], RadicalDepth[e],
66   Count[e, Power[_, _Rational], {0, Infinity}],
67   Total[bitCost /@ Cases[e, _Integer | _Rational | _Complex, {0, Infinity}]], LeafCount[e]
68 };
69 lessQ[a_, b_] := Module[{pairs, first},
70   pairs = Select[Transpose[{RadicalCost[a], RadicalCost[b]}], #[[1]] =!= #[[2]] &];
71   If[pairs === {}, False, first = First[pairs]; TrueQ[first[[1]] < first[[2]]]]
72 ];
73
74 canonicalNonzeroQ[e_] := e =!= 0 &&
75   MatchQ[e, _Integer | _Rational | _Complex | _Root | _AlgebraicNumber] &&
76   TrueQ[NumericQ[e] && Precision[e] === Infinity];
77 classify[e_] := Which[e === 0, "Equal", canonicalNonzeroQ[e], "Different", True, "Unknown"];
78 EqualityStatus[a_, b_, t_:5] := Module[{r},
79   If[!finiteNonnegativeQ[t] || TrueQ[t == 0], Return["Unknown"]];
80   r = TimeConstrained[
81     Block[{$Assumptions = True}, Quiet[Check[
82       If[ExactAlgebraicQ[a] && ExactAlgebraicQ[b], RootReduce[a - b], $Failed], $Failed]]],
83     t, $Failed];
84   classify[r]
85 ];
86 CertifiedEqualQ[a_, b_, t_:5] := EqualityStatus[a, b, t] === "Equal";
87
88 (* Public diagnostic helpers use their own time allowance. They are not called
89    from the engine, whose private stage[] shares one per-call deadline. *)
90 Factorc[e_, t_:5] := Module[{p},
91   If[!finiteNonnegativeQ[t] || TrueQ[t == 0], Return[e]];
92   p = TimeConstrained[Quiet[Check[If[ExactAlgebraicQ[e], Factor[e], e], e]], t, e];
93   If[CertifiedEqualQ[p, e, t], p, e]
```

```
];
rationalizeProposal[e_] := Module[{u, a, n, d, p, q, c},
  a = Together[e]; n = Numerator[a]; d = Denominator[a];
  If[d === 1, Return[e]];
  p = MinimalPolynomial[d, u];
  If[!PolynomialQ[p, u], Return[e]];
  c = p /. u -> 0;
  If[c === 0, Return[e]];
  q = PolynomialQuotient[p, u - d, u] /. u -> 0;
  Expand[-n q/c]
];
RationalizeDenominator[e_, t_:5] := Module[{p},
  If[!finiteNonnegativeQ[t] || TrueQ[t == 0], Return[e]];
  p = TimeConstrained[Quiet[Check[
    If[ExactAlgebraicQ[e], rationalizeProposal[e], e], e]], t, e];
  If[CertifiedEqualQ[p, e, t], p, e]
];

(* Resolve CURRENT defaults of the function actually called, not just explicit
   options. DeleteDuplicatesBy preserves the first explicit value, like the
   usual Wolfram option convention. Unknown names are rejected. *)
flattenOptions[items_List] := Flatten[(If[ListQ[#], flattenOptions[#], {#}] & /@ items), 1];
configuration[f_, explicit_List] := Module[{known, given, cfg, nonnegative, positive},
  given = flattenOptions[explicit];
  known = First /@ Options[f];
  If[!AllTrue[given, MatchQ[#, _Rule | _RuleDelayed] && MemberQ[known, First[#]] &],
    Return[$Failed]];
  cfg = Association[(First[#] -> Last[#] &) /@
    DeleteDuplicatesBy[Join[given, Options[f]], First]];
  If[!AllTrue[{"AllLevels", "Verbose", "Factor"}, MemberQ[{True, False}, cfg[#]] &], Return[
    $Failed]];
  nonnegative = {"MaxTrials", "MultiplierCap", "MaxRecords"};
  positive = {"MaxPasses", "MaxDegree", "MaxRootIndex", "MaxSolveDegree", "MaxLeafCount", "
    MaxBytes", "MemoryBudget"};
  If[!AllTrue[nonnegative, IntegerQ[cfg[#]] && cfg[#] >= 0 &], Return[$Failed]];
  If[!AllTrue[positive, IntegerQ[cfg[#]] && cfg[#] > 0 &], Return[$Failed]];
  If[!AllTrue[{"TimeBudget", "CertifyTime", "StageTime"}, finiteNonnegativeQ[cfg[#]] &], Return[
    $Failed]];
  If[cfg["MaxSolveDegree"] > 4, Return[$Failed]];
  If[!(cfg["Multipliers"] === Automatic || ListQ[cfg["Multipliers"]]), Return[$Failed]];
  If[!MatchQ[cfg["Solver"], Automatic | _Function | _Symbol], Return[$Failed]];
  cfg
];

(* Dynamic variables are private and localized by run[]. No persistent cache. *)
SetAttributes[stage, HoldAll];
stage[body_, limit_] := Module[{t, answer},
  t = Min[limit, $deadline - AbsoluteTime[]];
  If[!TrueQ[t > 0], $stageFailures++; Return[$Failed]];
  answer = TimeConstrained[Quiet[Check[body, $Failed]], t, $Failed];
  If[answer === $Failed, $stageFailures++]; answer
];
tick[] := If[AbsoluteTime[] >= $deadline, Throw[timeoutToken, $stopTag]];
withinBoundsQ[e_] := LeafCount[e] <= $cfg["MaxLeafCount"] && ByteCount[e] <= $cfg["MaxBytes"];
proof[a_, b_] := Module[{answer, status},
  answer = stage[If[ExactAlgebraicQ[a] && ExactAlgebraicQ[b], RootReduce[a - b], $Failed], $cfg["
    CertifyTime"]];
  status = classify[answer]; If[status === "Unknown", $unknown++]; status
];
accept[proposal_, original_, current_] := Module[{status},
```

```
  tick[];
  If[!withinBoundsQ[proposal] || !ExactAlgebraicQ[proposal] || !lessQ[proposal, current], Return[
    current]];
  status = proof[proposal, original];
  If[status =!= "Equal", Return[current]];
  $accepted++;
  If[Length[$records] < $cfg["MaxRecords"],
    AppendTo[$records, <|"Before" -> original, "After" -> proposal,
      "Method" -> "RootReduceDifference", "Residual" -> 0|>]];
  proposal
];

(* A local norm fast path. Both signs are proposals. This handles branch
   selection through the SAME acceptance gate, not by numerical sign tests. *)
quadraticProposals[e_] := Module[{a, b, c, z, rad, s, p, d, h, u, v, out = {}},
  If[Head[e] =!= Power || e[[2]] =!= 1/2, Return[{}]];
  rad = Expand[e[[1]]];
  s = DeleteDuplicates[Cases[rad, Power[q_?rationalQ, 1/2] :> Sqrt[q], {0, Infinity}]];
  If[Length[s] =!= 1, Return[{}]];
  s = First[s]; c = s^2;
  If[!rationalQ[c] || !TrueQ[c > 0], Return[{}]];
  p = rad /. s -> z;
  If[!PolynomialQ[p, z] || Exponent[p, z] =!= 1, Return[{}]];
  {a, b} = CoefficientList[p, z];
  If[!rationalQ[a] || !rationalQ[b], Return[{}]];
  d = Sqrt[a^2 - b^2 c];
  If[rationalQ[d],
    u = Sqrt[(a + d)/2]; v = Sqrt[(a - d)/2];
    out = Join[out, {u + v, u - v, -u + v, -u - v}]];
  h = Sqrt[b^2 - a^2/c];
  If[rationalQ[h],
    u = Sqrt[(b + h)/2]; v = Sqrt[(b - h)/2];
    out = Join[out, c^(1/4) {u + v, u - v, -u + v, -u - v}]];
  DeleteDuplicates[out]
];

reductionIndex[e_] := Module[{d, nodes},
  d = RadicalDepth[e];
  nodes = Select[Cases[e, Power[_, _Rational], {0, Infinity}], RadicalDepth[#] == d &];
  If[nodes === {}, 1, LCM @@ (Denominator[#[[2]]] & /@ nodes)]
];
multiplierSeeds[R_] := Module[{visible, seeds},
  If[ListQ[$cfg["Multipliers"]], Return[DeleteDuplicates[$cfg["Multipliers"]]]];
  visible = Cases[R, Power[b_?rationalQ, q_Rational] :> b^(1 - q), {0, Infinity}];
  seeds = Join[{1}, visible, Range[$cfg["MultiplierCap"]]];
  DeleteDuplicates[seeds]
];
smallRoots[g_, x_, d_] := Module[{c, sol},
  c = CoefficientList[g, x];
  Which[
    d == 1, {-c[[1]]/c[[2]]},
    d == 2, {(-c[[2]] + Sqrt[c[[2]]^2 - 4 c[[1]] c[[3]]])/(2 c[[3]]),
             (-c[[2]] - Sqrt[c[[2]]^2 - 4 c[[1]] c[[3]]])/(2 c[[3]])},
    d <= $cfg["MaxSolveDegree"],
      sol = Solve[g == 0, x]; If[ListQ[sol], x /. sol, {}],
    True, {}
  ]
];
gcdSearch[original_, initial_] := Module[
  {best = initial, r, R, seeds, m, x, p, g, degree, roots, root, k, candidate},
```

```
  r = reductionIndex[original];
  If[!IntegerQ[r] || r < 2 || r > $cfg["MaxRootIndex"] || $trials >= $cfg["MaxTrials"], Return[
    best]];
  R = stage[Expand[original^r], $cfg["StageTime"]];
  If[R === $Failed || !withinBoundsQ[R] || !ExactAlgebraicQ[R], Return[best]];
  seeds = multiplierSeeds[R];
  Do[
    tick[]; If[$trials >= $cfg["MaxTrials"], Break[]];
    (* Invalid seeds do not consume expensive trial slots, but are finite input. *)
    If[!withinBoundsQ[m] || !ExactAlgebraicQ[m] || proof[m, 0] =!= "Different", Continue[]];
    $trials++;
    p = stage[MinimalPolynomial[(m R)^(1/r), x], $cfg["StageTime"]];
    If[p === $Failed || !PolynomialQ[p, x] || !withinBoundsQ[p], Continue[]];
    If[Exponent[p, x] > $cfg["MaxDegree"], $degreeSkips++; Continue[]];
    g = stage[PolynomialGCD[p, x^r - m R, Extension -> Automatic], $cfg["StageTime"]];
    If[g === $Failed || !PolynomialQ[g, x] || !withinBoundsQ[g], Continue[]];
    degree = Exponent[g, x];
    If[!IntegerQ[degree] || degree < 1 || degree >= r || degree > $cfg["MaxSolveDegree"], Continue
    []];
    roots = stage[smallRoots[g, x, degree], $cfg["StageTime"]];
    If[!ListQ[roots], Continue[]];
    Do[
      If[!FreeQ[root, x] || !withinBoundsQ[root] || !ExactAlgebraicQ[root], Continue[]];
      Do[
        candidate = stage[Expand[root/m^(1/r) (-1)^(2 k/r)], $cfg["StageTime"]];
        best = accept[candidate, original, best], {k, 0, r - 1}],
      {root, roots}],
    {m, seeds}];
  best
];

(* No marker language and no ReplaceRepeated. Numerical islands are certified
   against their original value. Symbolic hosts are rebuilt only by congruence
   through List, Plus, Times and rational/integer Power. Other heads are opaque. *)
island[e_] := Module[{best = e, p, proposals, rebuilt},
  tick[];
  If[RadicalDepth[e] < 2 && FreeQ[e, _Root | _AlgebraicNumber], Return[e]];
  If[TrueQ[$cfg["AllLevels"]] && !AtomQ[e] && MemberQ[{Plus, Times, Power}, Head[e]],
    rebuilt = Map[walk, e]; best = accept[rebuilt, e, best]];
  If[$cfg["Solver"] =!= Automatic,
    p = stage[$cfg["Solver"][e], $cfg["StageTime"]];
    best = accept[p, e, best]];
  If[!FreeQ[e, _Root | _AlgebraicNumber],
    p = stage[ToRadicals[e], $cfg["StageTime"]]; best = accept[p, e, best]];
  proposals = stage[quadraticProposals[e], $cfg["StageTime"]];
  If[ListQ[proposals], Do[best = accept[p, e, best], {p, proposals}]];
  p = stage[RootReduce[e], $cfg["StageTime"]]; best = accept[p, e, best];
  If[TrueQ[$cfg["Factor"]], p = stage[Factor[e], $cfg["StageTime"]]; best = accept[p, e, best]];
  p = stage[Simplify[e, Assumptions -> True], $cfg["StageTime"]]; best = accept[p, e, best];
  If[RadicalDepth[best] >= 2, best = gcdSearch[e, best]];
  best
];
walk[e_] := Module[{h},
  tick[];
  If[!withinBoundsQ[e], Return[e]];
  If[ExactAlgebraicQ[e], Return[island[e]]];
  If[AtomQ[e], Return[e]];
  h = Head[e];
  If[MemberQ[{List, Plus, Times}, h] || (h === Power && rationalQ[e[[2]]]), Map[walk, e], e]
];
```

```
workflow[e_, coreOnly_] := Module[{best = e, next, passes, i},
  If[!withinBoundsQ[e], Return[e]];
  (* No promises about preservation of floating-point evaluation order. *)
  If[!FreeQ[e, _Real | _Complex?InexactNumberQ], Return[e]];
  If[coreOnly && !ExactAlgebraicQ[e], Return[e]];
  passes = If[TrueQ[$cfg["AllLevels"]], $cfg["MaxPasses"], 1];
  For[i = 1, i <= passes, i++,
    next = If[coreOnly, island[best], walk[best]];
    If[!lessQ[next, best], Break[]]; best = next];
  best
];
run[e_, cfg_, coreOnly_:False] := Module[{answer, status, inputCost, outputCost},
  If[cfg === $Failed, Return[<|"Value" -> e, "Status" -> "InvalidOptions"|>]];
  If[TrueQ[cfg["TimeBudget"] == 0], Return[<|"Value" -> e, "Status" -> "Disabled", "Trials" ->
    0|>]];
  Block[{$cfg = cfg, $deadline = AbsoluteTime[] + cfg["TimeBudget"],
    $stopTag = Unique["denestStop$"], $trials = 0, $stageFailures = 0,
    $unknown = 0, $accepted = 0, $degreeSkips = 0, $records = {}, $Assumptions = True},
    answer = Catch[TimeConstrained[
      MemoryConstrained[Module[{v = workflow[e, coreOnly]},
        {v, If[withinBoundsQ[e], RadicalCost[e], Missing["SizeLimit"]],
            If[withinBoundsQ[v], RadicalCost[v], Missing["SizeLimit"]]}],
        cfg["MemoryBudget"], memoryToken],
      cfg["TimeBudget"], timeoutToken], $stopTag];
    status = Which[answer === timeoutToken, "Timeout", answer === memoryToken, "MemoryLimit",
      SameQ[First[answer], e], "Unchanged", True, "Improved"];
    (* Atomic rollback: never return a partly rebuilt host after interruption. *)
    If[MemberQ[{"Timeout", "MemoryLimit"}, status],
      answer = e; inputCost = outputCost = Missing["Interrupted"],
      {answer, inputCost, outputCost} = answer];
    If[TrueQ[cfg["Verbose"]], Print["RadicalDenestImproved: ", status, "; multiplier trials = ",
    $trials]];
    <|"Value" -> answer, "Status" -> status, "Trials" -> $trials,
      "StageFailures" -> $stageFailures, "UnknownCertificates" -> $unknown,
      "DegreeSkips" -> $degreeSkips, "AcceptedLocalSteps" -> $accepted,
      "RecordsTruncated" -> ($accepted > Length[$records]), "LocalRecords" -> $records,
      "TrialLimitReached" -> ($trials >= cfg["MaxTrials"]),
      "InputCost" -> inputCost, "OutputCost" -> outputCost,
      "SearchIsComplete" -> False|>
  ]
];

Strad[e_, opts:OptionsPattern[]] := run[e, configuration[Strad, {opts}]]["Value"];
Strad[e_, all:(True|False), opts:OptionsPattern[]] :=
  run[e, configuration[Strad, Join[{"AllLevels" -> all}, {opts}]]]["Value"];
DenestRadicals[e_, opts:OptionsPattern[]] := run[e, configuration[DenestRadicals, {opts}]]["Value"
    ];
DenestRadicals[e_, all:(True|False), opts:OptionsPattern[]] :=
  run[e, configuration[DenestRadicals, Join[{"AllLevels" -> all}, {opts}]]]["Value"];
DenestCore[e_, opts:OptionsPattern[]] := run[e, configuration[DenestCore, {opts}], True]["Value"];
DenestReport[e_, opts:OptionsPattern[]] := run[e, configuration[DenestReport, {opts}]];

End[];
EndPackage[];
```

References

[1] V. Reshetnikov, *RadicalDenest: StradFixed.wl*, current branch URL supplied for review. Initial safety/wrapper portion inspected; remaining source unavailable. https://github.com/VladimirReshetnikov/RadicalDenest/blob/main/src/corrected/StradFixed.wl.

[2] V. Reshetnikov, *RadicalDenest, docs/report*. Current directory description inspected; earlier Library report used only as background. https://github.com/VladimirReshetnikov/RadicalDenest/tree/main/docs/report.

[3] V. Reshetnikov, *RadicalDenest, docs/literature*. Directory and download description. https://github.com/VladimirReshetnikov/RadicalDenest/tree/main/docs/literature.

[4] V. Reshetnikov, *RadicalDenest, src/code-review/unified*. Current directory description, including reported earlier Wolfram tests; report bodies and full logs not retrieved. https://github.com/VladimirReshetnikov/RadicalDenest/tree/main/src/code-review/unified.

[5] A. Borodin, R. Fagin, J. E. Hopcroft, and M. Tompa, "Decreasing the Nesting Depth of Expressions Involving Square Roots," *Journal of Symbolic Computation* **1** (1985), 169–188. https://www.cs.toronto.edu/~bor/Papers/decreasing-nesting-depth-for-expressions.pdf.

[6] E. Gkioulekas, "On the denesting of nested square roots," *International Journal of Mathematical Education in Science and Technology* **48**(6) (2017), 942–953. Author manuscript dated December 21, 2016. https://faculty.utrgv.edu/eleftherios.gkioulekas/papers/submitted/denesting-square-roots.pdf.

[7] D. J. Jeffrey and A. D. Rich, "Simplifying Square Roots of Square Roots by Denesting," in M. J. Wester (ed.), *Computer Algebra Systems: A Practical Guide*, Wiley, 1999, 61–72. https://cybertester.com/data/denest.pdf.

[8] M. Honsbeek, *Radical Extensions and Galois Groups*, Ph.D. thesis, Radboud University Nijmegen, 2005. In particular, Chapter 3, Theorem 104 and formula (3.5). https://www.math.ru.nl/~bosma/students/honsbeek/M_Honsbeek_thesis.pdf.

[9] S. Landau, *How to Tangle with a Nested Radical*, technical-report copy, University of Massachusetts, 1991. Historical exposition; not used to characterize the current state solely from its introductory wording. https://web.cs.umass.edu/publication/docs/1991/UM-CS-1991-076.pdf.

[10] Wolfram Research, *Power*, Wolfram Language documentation, accessed September 5, 2026. https://reference.wolfram.com/language/ref/Power.html.

[11] Wolfram Research, *RootReduce*, official documentation. https://reference.wolfram.com/language/ref/RootReduce.html.

[12] Wolfram Research, *PossibleZeroQ*, especially the "ExactAlgebraics" method. https://reference.wolfram.com/language/ref/PossibleZeroQ.html.

[13] Wolfram Research, *Simplify*, especially the assumptions and complexity options. https://reference.wolfram.com/language/ref/Simplify.html.

[14] Wolfram Research, *TimeConstrained*, official resource and interrupt semantics. https://reference.wolfram.com/language/ref/TimeConstrained.html.

[15] Wolfram Research, *MemoryConstrained*, official documentation. https://reference.wolfram.com/language/ref/MemoryConstrained.html.

[16] Wolfram Research, *Options*, official documentation. https://reference.wolfram.com/language/ref/Options.html.

[17] Wolfram Research, *PolynomialGCD*, especially `Extension -> Automatic`. https://reference.wolfram.com/language/ref/PolynomialGCD.html.

[18] Wolfram Research, *$MaxExtraPrecision*, especially hidden zeros and loss of requested precision. https://reference.wolfram.com/language/ref/\protect\T1\textdollarMaxExtraPrecision.html.